Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

A SKILL BASED EVALUATION REPORT

SUBMITTED BY

CHRISTOPHER NOBLE E.S(URK24CS7001)
ABISHEK K(URK24CS5071)
ASHISH MATHEW(URK24CS7033)
SHYAM SUNIL(URK24CS7057)

COURSE CODE
24AI2003

COURSE NAME

INTELLIGENT ROBOTICS

OCTOBER 2026

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

JARVIS: A Context-Aware, Voice-Controlled Agentic Virtual Assistant for Desktop Automation

Submitted by
CHRISTOPHER NOBLE E.S(URK24CS7001)
ABISHEK K(URK24CS5071)
ASHISH MATHEW(URK24CS7033)
SHYAM SUNIL(URK24CS7057)



DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956) Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

ABSTRACT

As personal computing environments grow increasingly complex, traditional desktop assistants remain limited by rigid command structures, lack of contextual awareness, and an inability to differentiate between local operating system environments and web-based resources. This project introduces an advanced, context-aware desktop assistant—designated as **JARVIS**—designed to bridge the gap between human intent and system automation.

JARVIS features a hybrid intelligence architecture that combines continuous speech recognition, real-time foreground window inspection via process monitoring, and a dual-intent execution engine capable of intelligently distinguishing between local executable software management and browser tab navigation. To overcome standard speech-to-text clipping and ambiguity errors, the system incorporates robust fallback matching and keyword-driven heuristics. Furthermore, the assistant integrates a dual-layer memory engine utilizing relational SQLite databases for preference scoring and execution logging, alongside a vector memory store for custom contextual recall. Supported by proactive background diagnostics, multi-step macro execution, and a reinforcement learning feedback loop, the system delivers a highly responsive, personalized, and secure autonomous workspace management solution.

PROBLEM STATEMENT

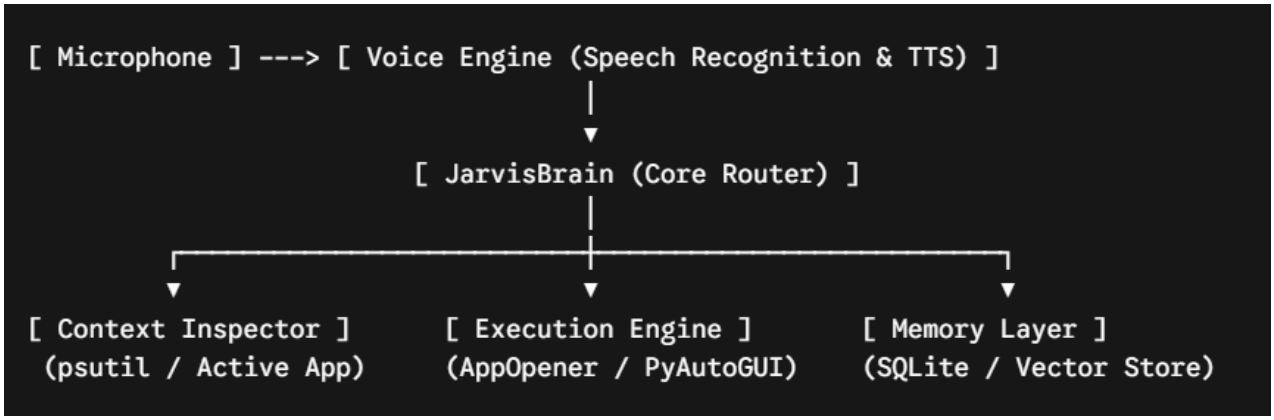
Modern personal computing environments require seamless multitasking between local operating system applications and web-based resources. However, existing desktop voice assistants suffer from critical limitations that hinder efficient workflow automation:

1. **Lack of Contextual Window Management:** Conventional assistants rely on rigid, hardcoded shortcuts that fail to distinguish between local desktop applications (requiring full process termination via `Alt + F4`) and web-browser tabs (requiring tab-level dismissal via `Ctrl + W`). This often results in unintended disruptions, such as closing an entire browser session instead of a single tab.
2. **Speech Recognition Vulnerability & Clipping:** Voice-to-text engines frequently clip initial command verbs (such as missing the word "open") or drop auxiliary terms due to audio latency or microphone noise, causing the assistant to fail or misinterpret standard requests.
3. **Absence of Foreground State Awareness:** Standard assistants operate blindly without tracking the active user workspace, leaving them unable to adapt their behavior based on what application or browser is currently in focus.
4. **Static Behavior and Fragmented Memory:** Most voice utilities lack persistent learning mechanisms, failing to track user command frequencies, adapt to execution feedback, or maintain a hybrid memory store for unstructured contextual recall.

Proposed Solution: To address these challenges, this project develops an intelligent, context-aware desktop automation assistant (**JARVIS**) equipped with real-time foreground process inspection, intent-driven dual execution logic for apps versus tabs, robust fallback matching for voice clipping, and a hybrid memory architecture (SQLite and vector stores) designed to deliver adaptive, personalized workspace management.

METHODOLOGY / ARCHITECTURE

The architecture of the JARVIS desktop assistant follows a modular, multi-threaded design pattern that decouples voice input/output processing, OS-level automation, context inspection, and data persistence. This ensures real-time responsiveness without blocking the main execution thread.



Core Modular Components

1. Voice Engine (`VoiceEngine`)

- **Speech Recognition:** Utilizes the Google Speech-to-Text API wrapped via the `speech_recognition` library with automated ambient noise adjustment (`adjust_for_ambient_noise`) to capture clear vocal inputs.
- **Thread-Safe Text-to-Speech (TTS):** Powered by `pyttsx3`, running on a background worker thread (`threading.Thread`) with `sapi5` (Windows) to enable concurrent speech playback without freezing execution loops.

2. Core Brain & Execution Engine (`JarvisBrain`)

- **Intent-Driven Routing:** Analyzes transcribed queries through a series of conditional checks and fallback parsers to determine user intent (e.g., opening local apps vs. browser tabs, executing macros, or system controls).
- **Context-Aware Window Inspection:** Interfaces with `psutil` to scan running active processes (`chrome.exe`, `code.exe`, etc.) in real-time, feeding foreground context into operational decisions.
- **Smart Universal Closing Logic:** Dynamically evaluates the active window state; triggers `Ctrl + W` if the user is in a web browser context or `Alt + F4` when a standalone desktop application is active.
- **Dual Opening Mechanism:** Separates local execution using `AppOpener` (for commands containing `"app"` or local software) from web routing via `webbrowser` (for browser tabs, URLs, or search queries).

3. Hybrid Memory & Persistence Layer

- **Relational Database (`sqlite3`):** Manages `memory.db` to store command execution logs (`command_logs`), track user preference frequencies, and maintain dynamic feedback scoring for reinforcement learning.

- **Vector Memory Store (`vector_store.json`):** Implements a lightweight keyword-overlap vector search engine (`VectorMemory`) to store, categorize, and recall custom user facts and snippets dynamically.

4. Proactive Background Diagnostics (`ProactiveScheduler`)

- Runs as a daemon thread in the background, periodically polling system metrics via `psutil`.
- Triggers vocal alerts and proactive interventions if hardware thresholds (such as CPU usage exceeding 80%) are breached.

Execution Workflow

1. **Audio Capture:** The microphone captures the user's voice; the `VoiceEngine` filters background noise and converts speech into a lowercase query string.
2. **Context Update:** `JarvisBrain` updates current system parameters, timestamps, and active window handles using `psutil`.
3. **Command Parsing & Fallback Matching:** The query is evaluated against keyword rules and fallback triggers to handle potential audio clipping or missing auxiliary words.
4. **Action Execution:** Depending on the intent, the system either:
    - Fires OS key-combinations (`pyautogui`),
    - Launches local binaries (`AppOpener` / `subprocess`), or
    - Opens URLs (`webbrowser`).
5. **Logging & Reinforcement:** The executed action and timestamp are logged into `memory.db`, updating preference frequencies for future adaptive optimization.

IMPLEMENTATION – CODING AND OUTPUT SCREENSHOT

Brain.py:

```
import os
import sys
import time
import glob
import json
import sqlite3
import datetime
import threading
import subprocess
import webbrowser
import pyautogui
import psutil
import AppOpener
import numpy as np
import speech_recognition as sr
import pyttsx3

DB_FILE = "memory.db"
VECTOR_FILE = "vector_store.json"
LOG_FILE = "logs.txt"

# ========================================
# 1. DATABASE & MEMORY MANAGEMENT
# ========================================
def init_db():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS long_term_memory (
            key TEXT PRIMARY KEY,
            value TEXT,
            updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS command_logs (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            command TEXT,
            action_taken TEXT,
            timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
        )
    ''')
    cursor.execute('''
        CREATE TABLE IF NOT EXISTS user_preferences (
            pref_key TEXT PRIMARY KEY,
            pref_value TEXT,
            frequency INTEGER DEFAULT 1,
            feedback_score REAL DEFAULT 1.0
        )
    ''')
    conn.commit()
    conn.close()

init_db()

# ==========================================
# 2. VECTOR MEMORY ENGINE
# ==========================================
class VectorMemory:
    def __init__(self):
        self.memories = []
        self.load_memories()

    def load_memories(self):
        if os.path.exists(VECTOR_FILE):
            try:
                with open(VECTOR_FILE, "r") as f:
                    self.memories = json.load(f)
            except Exception:
                self.memories = []

    def save_memories(self):
        with open(VECTOR_FILE, "w") as f:
            json.dump(self.memories, f, indent=2)

    def add_memory(self, text, category="general"):
        self.memories.append({
            "text": text,
            "category": category,
            "timestamp": str(datetime.datetime.now())
        })
        self.save_memories()

    def search_memory(self, query):
        query_words = set(query.lower().split())
        best_match = None
```

```
        max_overlap = 0
        for item in self.memories:
            text_words = set(item["text"].lower().split())
            overlap = len(query_words.intersection(text_words))
            if overlap > max_overlap:
                max_overlap = overlap
                best_match = item["text"]
        return best_match

# ==========================================
# 3. VOICE ENGINE (THREAD-SAFE CONTINUOUS TTS)
# ==========================================
class VoiceEngine:
    def __init__(self, log_callback=None):
        self.log_callback = log_callback
        self.recognizer = sr.Recognizer()

    def speak(self, text):
        print(f"JARVIS: {text}")
        if self.log_callback:
            self.log_callback(f"JARVIS: {text}")

        def tts_worker(phrase):
            try:
                engine = pyttsx3.init('sapi5' if os.name == 'nt' else 'espeak')
                voices = engine.getProperty('voices')
                if voices:
                    engine.setProperty('voice', voices[0].id)
                engine.setProperty('rate', 185)
                engine.say(phrase)
                engine.runAndWait()
            except Exception as e:
                print(f"[TTS Error]: {e}")

        speaker_thread = threading.Thread(target=tts_worker, args=(text,))
        speaker_thread.start()
        speaker_thread.join()

    def listen(self):
        with sr.Microphone() as source:
            self.recognizer.adjust_for_ambient_noise(source, duration=0.4)
            if self.log_callback:
                self.log_callback("[Listening...]")
            try:
                audio = self.recognizer.listen(source, timeout=5, phrase_time_limit=8)
                if self.log_callback:
```

```
                self.log_callback("[Understanding...]")
            query = self.recognizer.recognize_google(audio, language='en-US')
            return query.lower().strip()
        except Exception:
            return ""

# ==========================================
# 4. ADVANCED BRAIN & OS AUTOMATION
# ==========================================
class JarvisBrain:
    def __init__(self, voice_engine, ui_updater=None):
        self.voice = voice_engine
        self.ui_updater = ui_updater
        self.vector_mem = VectorMemory()
        self.pending_feedback_command = None
        self.context = {
            "active_app": "Unknown",
            "last_command": "None",
            "recent_commands": [],
            "active_task": "Idle",
            "current_time": "",
            "current_date": ""
        }
        self.update_context()

    def detect_active_application(self):
        try:
            for proc in psutil.process_iter(['name']):
                if proc.info['name'] in ['chrome.exe', 'code.exe', 'msedge.exe',
'notepad.exe', 'python.exe', 'spotify.exe', 'VALORANT-Win64-Shipping.exe']:
                    return proc.info['name']
            return "Desktop / Explorer"
        except Exception:
            return "System Workspace"

    def update_context(self):
        now = datetime.datetime.now()
        self.context["current_time"] = now.strftime("%I:%M %p")
        self.context["current_date"] = now.strftime("%A, %B %d, %Y")
        self.context["active_app"] = self.detect_active_application()

    def log_execution(self, command, action):
        self.context["last_command"] = command
        self.context["recent_commands"].append(command)
        if len(self.context["recent_commands"]) > 5:
            self.context["recent_commands"].pop(0)
```

```
        with open(LOG_FILE, "a") as f:
            f.write(f"[{datetime.datetime.now()}] CMD: '{command}' | ACTION:
'{action}'\n")

        conn = sqlite3.connect(DB_FILE)
        cursor = conn.cursor()
        cursor.execute("INSERT INTO command_logs (command, action_taken)
VALUES (?, ?)", (command, action))
        cursor.execute("SELECT frequency FROM user_preferences WHERE
pref_key = ?", (command,))
        row = cursor.fetchone()
        if row:
            cursor.execute("UPDATE user_preferences SET frequency = frequency + 1
WHERE pref_key = ?", (command,))
        else:
            cursor.execute("INSERT INTO user_preferences (pref_key, pref_value,
frequency, feedback_score) VALUES (?, ?, 1, 1.0)", (command, action))
        conn.commit()
        conn.close()

        if self.ui_updater:
            self.ui_updater()

    def confirm_action(self, warning_message):
        self.voice.speak(f"Hold on boss. {warning_message}. Are you sure you want
me to proceed? Say yes or no.")
        response = self.voice.listen()
        if "yes" in response or "confirm" in response or "do it" in response:
            return True
        self.voice.speak("Got it. Aborting action.")
        return False

    def process_feedback(self, command, is_positive):
        score_change = 0.5 if is_positive else -0.5
        conn = sqlite3.connect(DB_FILE)
        cursor = conn.cursor()
        cursor.execute("UPDATE user_preferences SET feedback_score =
feedback_score + ? WHERE pref_key = ?", (score_change, command))
        conn.commit()
        conn.close()
        if is_positive:
            self.voice.speak("Awesome! I'll remember you like it this way.")
        else:
            self.voice.speak("Noted. I will adjust how I handle that next time.")
```

```
    def execute_command(self, query):
        if not query:
            return

        # Clean wake word from the start
        query = query.replace("jarvis", "").strip()

        self.update_context()
        self.context["active_task"] = f"Executing: '{query}'"

        # Reinforcement Feedback
        if query in ["that was good", "nice job", "good work"]:
            if self.pending_feedback_command:
                self.process_feedback(self.pending_feedback_command, True)
                self.pending_feedback_command = None
            return
        elif query in ["bad action", "wrong", "don't do that"]:
            if self.pending_feedback_command:
                self.process_feedback(self.pending_feedback_command, False)
                self.pending_feedback_command = None
            return

        # Study Mode Macro
        elif "start my study session" in query or "study mode" in query:
            self.voice.speak("Setting up your study environment right away, boss!")
            webbrowser.open("https://google.com")
            webbrowser.open("https://scholar.google.com")
            subprocess.Popen(["notepad.exe" if os.name == 'nt' else 'gedit'])
            self.log_execution(query, "Macro: Study Environment Launched")
            self.pending_feedback_command = query

        # Context Awareness
        elif "what am i doing" in query or "current context" in query:
            app = self.context["active_app"]
            self.voice.speak(f"You're currently working in {app}. Time is
{self.context['current_time']}.")
            self.log_execution(query, "Provided Context Status")

        # Vector Memory Integration
        elif "remember that" in query:
            fact = query.replace("remember that", "").strip()
            self.vector_mem.add_memory(fact, category="user_preference")
            self.voice.speak(f"Locked into memory: {fact}")
            self.log_execution(query, "Stored in Vector Memory")
        elif "what do you remember about" in query or "recall" in query:
            search_term = query.replace("what do you remember about",
```

```
"").replace("recall", "").strip()
            result = self.vector_mem.search_memory(search_term)
            if result:
                self.voice.speak(f"Here's what I found: {result}")
            else:
                self.voice.speak("I couldn't find any related memory fragments on that
topic.")
            self.log_execution(query, "Queried Vector Memory")

        # Windows Settings
        elif "open settings" in query or "settings" in query:
            self.voice.speak("Opening Windows Settings for you, boss.")
            if os.name == 'nt':
                os.system("start ms-settings:")
            self.log_execution(query, "Opened Windows Settings")

        # Valorant Launcher
        elif "open val" in query or "open valorant" in query:
            self.voice.speak("Firing up Valorant! Good luck in your match, boss.")
            if os.name == 'nt':
                valorant_path = r"C:\Riot Games\Riot Client\RiotClientServices.exe"
                if os.path.exists(valorant_path):
                    subprocess.Popen([valorant_path, "--launch-product=valorant", "--
launch-patchline=live"])
                else:
                    os.system("start valorant")
            self.log_execution(query, "Launched Valorant")

        # System File Search
        elif "find file" in query or "search file" in query:
            filename = query.replace("find file", "").replace("search file", "").strip()
            self.voice.speak(f"Searching your system for {filename}...")
            user_path = os.path.expanduser("~")
            found_files = glob.glob(f"{user_path}/**/*{filename}*", recursive=True)
            if found_files:
                target = found_files[0]
                self.voice.speak(f"Found it! Opening {os.path.basename(target)}")
                os.startfile(target)
                self.log_execution(query, f"Opened File: {target}")
            else:
                self.voice.speak(f"Sorry boss, couldn't locate any file named
{filename}.")

        # CONTEXT-AWARE SMART CLOSING LOGIC (WITH TARGETED
TAB CLOSING)
        elif query.startswith("close"):
```

```
            target_to_close = query.replace("close", "").strip().lower()
            active_app = self.detect_active_application().lower()

            if target_to_close and target_to_close not in ["tab", "app", "window"]:
                self.voice.speak(f"Closing {target_to_close}, boss.")
                if "chrome" in active_app or "edge" in active_app or "browser" in
active_app:
                    pyautogui.hotkey("ctrl", "shift", "a")  # Open tab search dropdown
                    time.sleep(0.3)
                    pyautogui.write(target_to_close, interval=0.05)
                    time.sleep(0.4)
                    pyautogui.press("enter")
                    time.sleep(0.3)
                    pyautogui.hotkey("ctrl", "w")  # Close that specific tab
                else:
                    pyautogui.hotkey("alt", "f4")
                self.log_execution(query, f"Targeted Close: {target_to_close}")
            else:
                if "chrome" in active_app or "edge" in active_app or "browser" in
active_app:
                    self.voice.speak("Closing current browser tab, boss.")
                    pyautogui.hotkey("ctrl", "w")
                    self.log_execution(query, "Closed Browser Tab (Ctrl+W)")
                else:
                    self.voice.speak("Closing application, boss.")
                    pyautogui.hotkey("alt", "f4")
                    self.log_execution(query, "Closed Desktop App (Alt+F4)")

        elif "switch window" in query or "next window" in query:
            pyautogui.hotkey("alt", "tab")
            self.log_execution(query, "Switched Window")

        elif "type " in query:
            text_to_type = query.replace("type ", "").strip()
            pyautogui.write(text_to_type, interval=0.05)
            self.log_execution(query, f"Typed text: {text_to_type}")

        # Hardware Volume Control
        elif "volume up" in query:
            pyautogui.press("volumeup", presses=5)
            self.voice.speak("Turned volume up, boss.")
            self.log_execution(query, "Volume Up")
        elif "volume down" in query:
            pyautogui.press("volumedown", presses=5)
            self.voice.speak("Turned volume down, boss.")
            self.log_execution(query, "Volume Down")
```

```
        elif "mute" in query:
            pyautogui.press("volumemute")
            self.voice.speak("Muted audio.")
            self.log_execution(query, "Muted Audio")

        # Terminal Commands
        elif "run command" in query or "terminal " in query:
            cmd = query.replace("run command", "").replace("terminal", "").strip()
            if self.confirm_action(f"Execute terminal command '{cmd}'"):
                self.voice.speak("Executing terminal command.")
                result = subprocess.getoutput(cmd)
                print(f"Command Output:\n{result}")
                self.log_execution(query, f"Executed CMD: {cmd}")

        elif "delete file" in query or "purge system" in query:
            if self.confirm_action("This involves deleting sensitive files"):
                self.voice.speak("Executing request under safety override.")
                self.log_execution(query, "File Action Executed (Confirmed)")

        # Conversational / Chit-Chat
        elif any(g in query for g in ["hello", "hi", "hey"]):
            self.voice.speak("Hey there! Ready to build something great today, boss?")
            self.log_execution(query, "Friendly Greeting")
        elif "how are you" in query:
            self.voice.speak("All systems running at peak efficiency! How are you doing
today?")
            self.log_execution(query, "Chitchat Response")

        # ========================================
        # INTENT-BASED OPEN HANDLER (App vs Tab/Web)
        # ========================================
        elif "open " in query or any(site in query for site in ["w3schools", "classroom",
"wiki", "youtube", "github", "stackoverflow", "spotify", "notepad", "chrome",
"calendar", "comet", "chatgpt", "chat gpt", "gpt", "drive", "maps", "docs", "meet"]):
            if query.startswith("open "):
                target = query.replace("open ", "").strip()
            else:
                target = query.strip()

            wants_local_app = "app" in target
            wants_web_tab = "tab" in target or "website" in target or "site" in target

            target = target.replace(" app", "").replace(" tab", "").replace(" website",
"").replace(" site", "").replace(" in google", "").replace(" on google", "").strip()

            WEB_ALIASES = {
```

```
    "chatgpt": "https://chatgpt.com",
    "chat gpt": "https://chatgpt.com",
    "gpt": "https://chatgpt.com",
    "openai": "https://chatgpt.com",
    "youtube": "https://www.youtube.com",
    "github": "https://www.github.com",
    "stackoverflow": "https://stackoverflow.com",
    "w3schools": "https://www.w3schools.com",
    "classroom": "https://classroom.google.com",
    "google classroom": "https://classroom.google.com",
    "gcr": "https://classroom.google.com",
    "drive": "https://drive.google.com",
    "google drive": "https://drive.google.com",
    "maps": "https://maps.google.com",
    "google maps": "https://maps.google.com",
    "docs": "https://docs.google.com",
    "google docs": "https://docs.google.com",
    "meet": "https://meet.google.com",
    "google meet": "https://meet.google.com",
    "wiki": "https://www.wikipedia.org",
    "wikipedia": "https://www.wikipedia.org"
}

if target in ["chrome", "google chrome"]:
    self.voice.speak("Opening Google Chrome, boss!")
    try:
        AppOpener.open("google chrome", match_closest=True)
    except Exception:
        webbrowser.open("https://www.google.com")
    self.log_execution(query, "Opened Chrome")

elif target in WEB_ALIASES:
    self.voice.speak(f"Opening {target} in your browser, boss!")
    webbrowser.open(WEB_ALIASES[target])
    self.log_execution(query, f"Opened Web URL: {target}")

elif wants_web_tab or ("." in target and " " not in target):
    self.voice.speak(f"Opening {target} in browser, boss!")
    if "." in target and " " not in target:
        url = target if target.startswith("http") else f"https://{target}"
        webbrowser.open(url)
    else:
        webbrowser.open(f"https://www.google.com/search?q={target}")
    self.log_execution(query, f"Opened Web Tab: {target}")

elif wants_local_app:
```

```
            self.voice.speak(f"Opening {target} application, boss!")
            try:
                AppOpener.open(target, match_closest=True)
                self.log_execution(query, f"Opened Local App: {target}")
            except Exception:
                webbrowser.open(f"https://www.google.com/search?q={target}")
                self.log_execution(query, f"Fallback Web Search: {target}")

        else:
            self.voice.speak(f"Opening {target} for you, boss!")
            try:
                AppOpener.open(target, match_closest=True)
                self.log_execution(query, f"Opened App (Default): {target}")
            except Exception:
                webbrowser.open(f"https://www.google.com/search?q={target}")
                self.log_execution(query, f"Fallback Web Search: {target}")

    else:
        self.voice.speak("I hear you, boss, but I'm not sure how to handle that
command yet.")
        self.log_execution(query, "Unknown Command")

    self.context["active_task"] = "Idle"
    if self.ui_updater:
        self.ui_updater()

# ==========================================
# 5. PROACTIVE SCHEDULER
# ==========================================
class ProactiveScheduler(threading.Thread):
    def __init__(self, brain, voice):
        super().__init__()
        self.brain = brain
        self.voice = voice
        self.daemon = True

    def run(self):
        while True:
            now = datetime.datetime.now()
            if now.minute == 0:
                cpu = psutil.cpu_percent()
                if cpu > 80:
                    self.voice.speak(f"Proactive Alert: Hey boss, CPU usage is high at
{cpu} percent. Want me to close background apps?")
                    self.brain.log_execution("PROACTIVE_CHECK", "High CPU
Warning")
```

```
            time.sleep(45)

if __name__ == "__main__":
    print("Initializing JARVIS Core Backend...")
    voice_engine = VoiceEngine()
    brain = JarvisBrain(voice_engine)
    scheduler = ProactiveScheduler(brain, voice_engine)
    scheduler.start()

    voice_engine.speak("All systems online. Standing by for commands, boss.")
    while True:
        command = voice_engine.listen()
        if command:
            print(f"[Captured Voice]: '{command}'")
            brain.execute_command(command)
```

Gui.py:

```
import sys
import sqlite3
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget,
QVBoxLayout,
                             QHBoxLayout, QLabel, QTextEdit, QListWidget, QPushButton,
QStackedWidget)
from PyQt5.QtCore import QTimer, Qt, pyqtSignal, QObject
from PyQt5.QtGui import QFont

class GuiSignalBridge(QObject):
    update_signal = pyqtSignal()

class JarvisDashboard(QMainWindow):
    def __init__(self, brain):
        super().__init__()
        self.brain = brain
        self.bridge = GuiSignalBridge()
        self.bridge.update_signal.connect(self.refresh_dashboard)

        self.initUI()
        self.timer = QTimer()
        self.timer.timeout.connect(self.refresh_dashboard)
        self.timer.start(1000)

    def initUI(self):
        self.setWindowTitle("Jarvis AI")
```

```
        self.setGeometry(100, 100, 950, 600)
        self.setStyleSheet("""
            QMainWindow { background-color: #05070a; }
            QLabel { color: #d0e0f0; font-family: Segoe UI; }
            QPushButton {
                background-color: #0d1522;
                color: #00f0ff;
                border: 1px solid #1a3550;
                border-radius: 6px;
                padding: 6px 14px;
                font-weight: bold;
                font-family: Segoe UI;
            }
            QPushButton:hover { background-color: #142238; border: 1px solid
#00f0ff; }
            QTextEdit, QListWidget {
                background-color: #030508;
                color: #00ffaa;
                border: 1px solid #142840;
                border-radius: 6px;
                font-family: Consolas, Monospace;
            }
        """)

        central_widget = QWidget()
        self.setCentralWidget(central_widget)
        main_layout = QVBoxLayout(central_widget)
        main_layout.setContentsMargins(15, 15, 15, 15)

        # Top Navigation Bar
        top_bar = QHBoxLayout()
        title_lbl = QLabel("Jarvis AI")
        title_lbl.setFont(QFont("Segoe UI", 12, QFont.Bold))
        title_lbl.setStyleSheet("color: #ffffff;")
        top_bar.addWidget(title_lbl)

        top_bar.addStretch()

        self.btn_home = QPushButton("🏠 Home")
        self.btn_home.clicked.connect(lambda: self.switch_page(0))
        top_bar.addWidget(self.btn_home)

        self.btn_logs = QPushButton("💬 Logs & Memory")
        self.btn_logs.clicked.connect(lambda: self.switch_page(1))
        top_bar.addWidget(self.btn_logs)
```

```
main_layout.addLayout(top_bar)

# Stacked Pages
self.stack = QStackedWidget()

# --- PAGE 0: Clean Home Screen ---
home_page = QWidget()
home_layout = QVBoxLayout(home_page)
home_layout.setAlignment(Qt.AlignCenter)

self.orb_lbl = QLabel("◍")
self.orb_lbl.setFont(QFont("Segoe UI", 60))
self.orb_lbl.setAlignment(Qt.AlignCenter)
home_layout.addWidget(self.orb_lbl)

self.status_lbl = QLabel("Available...")
self.status_lbl.setFont(QFont("Segoe UI", 12))
self.status_lbl.setStyleSheet("color: #00f0ff;")
self.status_lbl.setAlignment(Qt.AlignCenter)
home_layout.addWidget(self.status_lbl)

# Interactive Mic Button
mic_btn = QPushButton(" 🎙 □")
mic_btn.setFixedSize(55, 55)
mic_btn.setStyleSheet("""
    QPushButton {
        background-color: #007acc;
        color: white;
        border-radius: 27px;
        font-size: 20px;
        border: none;
    }
    QPushButton:hover { background-color: #0098ff; }
""")
mic_btn.clicked.connect(self.trigger_manual_listen)

mic_container = QHBoxLayout()
mic_container.setAlignment(Qt.AlignCenter)
mic_container.addWidget(mic_btn)
home_layout.addLayout(mic_container)

self.stack.addWidget(home_page)

# --- PAGE 1: Simple Logs & Memory View ---
logs_page = QWidget()
logs_layout = QHBoxLayout(logs_page)
```

```
        left_pane = QVBoxLayout()
        lbl_prefs_title = QLabel("Learned Preferences")
        lbl_prefs_title.setStyleSheet("color: #00f0ff; font-weight: bold;")
        left_pane.addWidget(lbl_prefs_title)

        self.lst_prefs = QListWidget()
        left_pane.addWidget(self.lst_prefs)
        logs_layout.addLayout(left_pane, stretch=1)

        right_pane = QVBoxLayout()
        lbl_logs_title = QLabel("System Audit Trail")
        lbl_logs_title.setStyleSheet("color: #00f0ff; font-weight: bold;")
        right_pane.addWidget(lbl_logs_title)

        self.txt_logs = QTextEdit()
        self.txt_logs.setReadOnly(True)
        right_pane.addWidget(self.txt_logs)
        logs_layout.addLayout(right_pane, stretch=2)

        self.stack.addWidget(logs_page)
        main_layout.addWidget(self.stack)

    def switch_page(self, index):
        self.stack.setCurrentIndex(index)

    def set_status(self, text):
        self.status_lbl.setText(text)

    def trigger_manual_listen(self):
        self.set_status("Listening...")
        self.brain.voice.speak("I'm listening, boss!")
        self.set_status("Available...")

    def trigger_ui_update(self):
        self.bridge.update_signal.emit()

    def refresh_dashboard(self):
        self.lst_prefs.clear()
        try:
            conn = sqlite3.connect("memory.db")
            cursor = conn.cursor()
            cursor.execute("SELECT pref_key, frequency FROM user_preferences
ORDER BY frequency DESC LIMIT 15")
            for key, freq in cursor.fetchall():
                self.lst_prefs.addItem(f"{key} (Uses: {freq})")
```

```
            conn.close()
        except Exception:
            pass

        try:
            with open("logs.txt", "r") as f:
                lines = f.readlines()[-25:]
                self.txt_logs.setText("".join(lines))
        except FileNotFoundError:
            self.txt_logs.setText("System initializing...")
```

Main.py:

```
import sys
import time
import threading
from PyQt5.QtWidgets import QApplication
from brain import VoiceEngine, JarvisBrain, ProactiveScheduler
from gui import JarvisDashboard

class JarvisApp:
    def __init__(self):
        self.qt_app = QApplication(sys.argv)
        self.voice = VoiceEngine()
        self.brain = JarvisBrain(self.voice)
        self.gui = JarvisDashboard(self.brain)

        self.brain.ui_updater = self.gui.trigger_ui_update
        self.is_active = True  # Tracks wake/sleep state

        self.proactive_thread = ProactiveScheduler(self.brain, self.voice)
        self.proactive_thread.start()

        self.listen_thread = threading.Thread(target=self.voice_loop, daemon=True)
        self.listen_thread.start()

    def voice_loop(self):
        self.voice.speak("JARVIS system active, boss! I'm listening. What are we
doing today?")

        while True:
            try:
                query = self.voice.listen()

                if not query or query.strip() == "":
```

```
                time.sleep(0.1)
                continue

            print(f"\n[Captured Voice]: '{query}'")

            # If sleeping, check for wake words
            if not self.is_active:
                if any(wake in query for wake in ["wake up", "hey jarvis", "jarvis"]):
                    self.is_active = True
                    self.voice.speak("I'm awake and ready, boss!")
                    if self.gui:
                        self.gui.set_status("Available...")

                    clean_query = query.replace("hey jarvis", "").replace("wake up",
"").replace("jarvis", "").strip()
                    if clean_query:
                        self.brain.execute_command(clean_query)
                continue

            # Check for sleep / standby command
            if any(stop_word in query for stop_word in ["stop listening", "go to
sleep", "shut down jarvis", "bye jarvis"]):
                self.voice.speak("Going on standby. Say 'Wake up' or 'Hey Jarvis'
whenever you need me.")
                self.is_active = False
                if self.gui:
                    self.gui.set_status("On Standby (Say 'Wake up')")
                continue

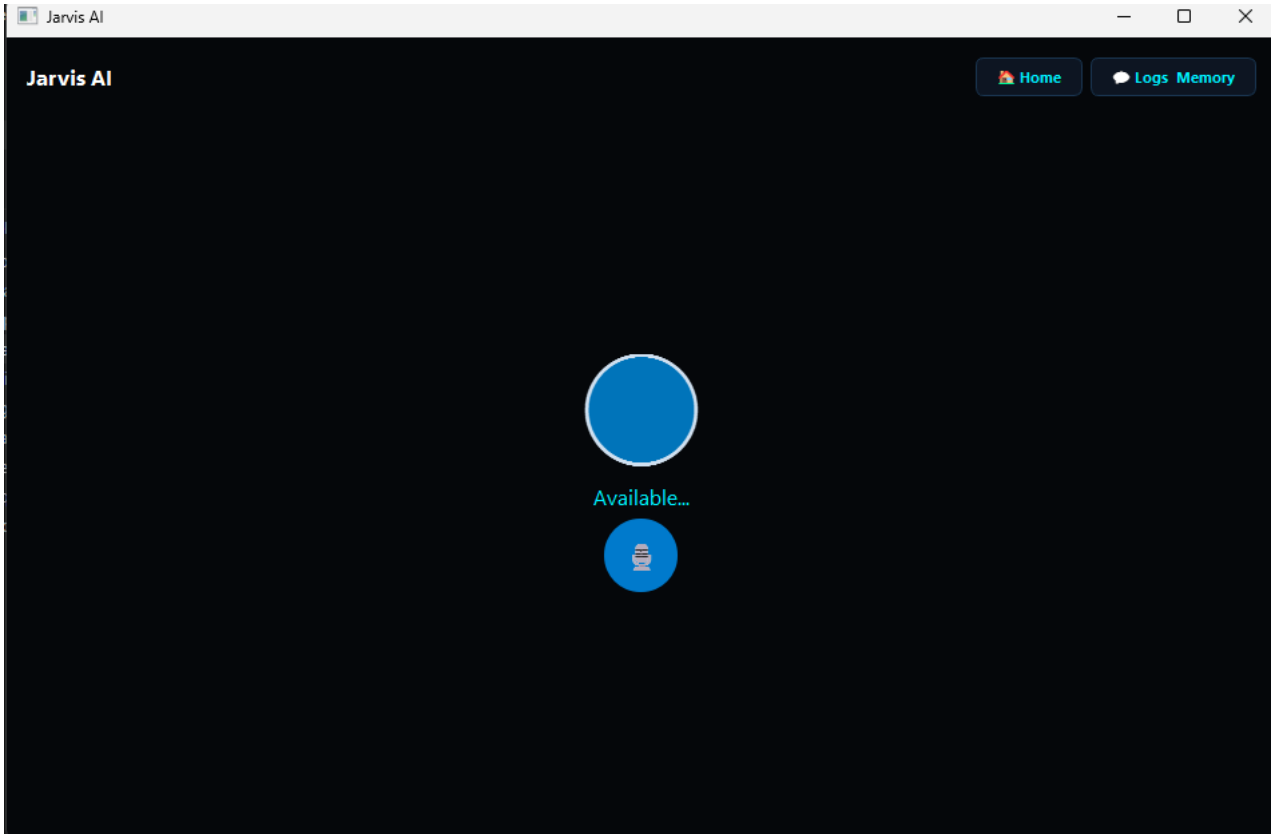
            # Execute active command normally
            self.brain.execute_command(query)

        except Exception as e:
            print(f"[Recovered Loop Error]: {e}")
            time.sleep(0.5)

    def run(self):
        self.gui.show()
        sys.exit(self.qt_app.exec_())

if __name__ == "__main__":
    app = JarvisApp()
    app.run()
```

Output:




```
[Captured Voice]: 'open chat gpt'
JARVIS: Opening chat gpt in your browser, boss!

[Captured Voice]: 'open gemini'
JARVIS: Opening gemini for you, boss!
GEMINI NOT FOUND... TYPE (LS) for list of applications.

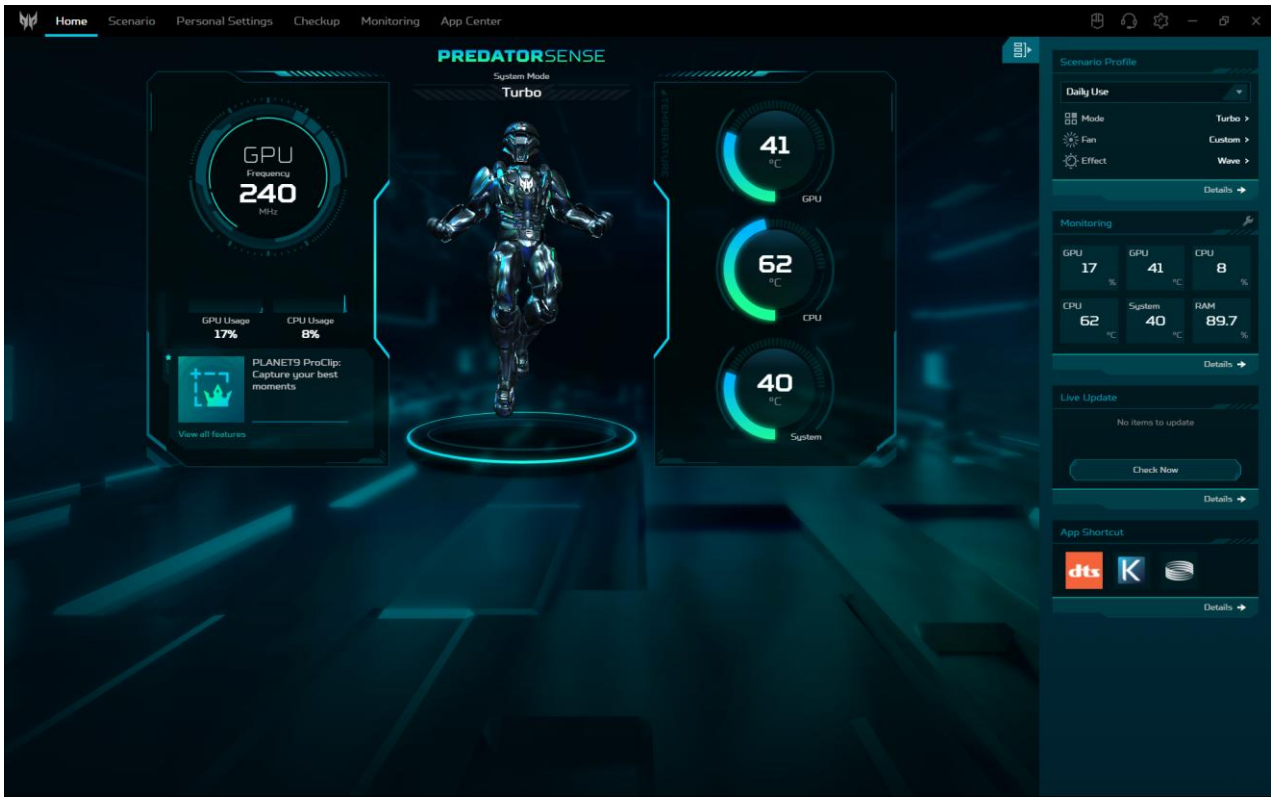
[Captured Voice]: 'open predator sense'
JARVIS: Opening predator sense for you, boss!

Closest match to PREDATOR SENSE : ['predatorsense']
Try : [open("predator sense", match_closest=True)]

OPENING PREDATORSENSE

[Captured Voice]: 'youtube'
JARVIS: Opening youtube in your browser, boss!

[Captured Voice]: 'close youtube'
JARVIS: Closing youtube, boss.
PS C:\Users\noble\OneDrive\Desktop\Intelliget robo 4 pro>
```


Home
Scenario
Personal Settings
Checkup
Monitoring
App Center
PREDATORSENSE
System Mode
Turbo
GPU
Frequency
240
MHz
GPU Usage
17%
CPU Usage
8%
PLANET9 ProClip:
Capture your best
moments
View all features
41
°C
GPU
62
°C
CPU
40
°C
System
Scenario Profile
Daily Use
Mode
Turbo
Fan
Custom
Effect
Wave
Details
Monitoring
GPU
17
%
GPU
41
°C
CPU
8
%
CPU
62
°C
System
40
°C
RAM
89.7
%
Details
Live Update
No items to update
Check Now
Details
App Shortcut
Details

CONCLUSION

This project successfully developed and implemented **JARVIS**, an advanced, context-aware desktop automation assistant designed to bridge the gap between human intent and system-level execution. By integrating real-time foreground window inspection (`psutil`) with an intent-driven dual execution engine, the system eliminates the rigid limitations of conventional voice assistants—seamlessly differentiating between local application management (`Alt + F4`) and browser tab navigation (`Ctrl + W`).

Furthermore, the integration of robust keyword fallbacks, multi-step macro execution, proactive CPU health monitoring, and a hybrid memory architecture (SQLite relational logging and vector-based snippet retrieval) ensures both operational resilience and adaptive personalization. The resulting framework provides a highly responsive, secure, and intelligent workspace assistant, paving the way for more autonomous, context-driven human-computer interaction in modern personal computing environments.

EVALUATION SHEET

Reg .No :URK24CS7001

Name :E.S. CHRISTOPHER NOBLE

Course code: 24AI2003

Course Name: INTELLIGENT ROBOTICS

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Simulation Activities	10	
2	Intelligent Robotics Project	30	
Total		40	

S. No.	Rubrics	Maximum Marks	Marks Obtained
1	Robot Design and Architecture	5	
2	Kinematics and Motion Planning	5	
3	AI / ML Algorithm Implementation	5	
4	Sensor Integration and Perception	5	
5	Robot Control and Navigation	5	
6	Innovation and Critical Thinking	4	
7	Performance Analysis and Optimization	4	
8	Presentation and Viva Voce	4	
9	Completeness of the Report	3	
	Total	40	

Performance-Level Assessment

Use the following performance levels to evaluate each rubric.

Rubrics	Excellent	Good	Average	Below Average
Robot Design and Architecture (5)	5	4	3	0–2
Kinematics and Motion Planning (5)	5	4	3	0–2
AI / ML Algorithm Implementation (5)	5	4	3	0–2
Sensor Integration and Perception (5)	5	4	3	0–2
Robot Control and Navigation (5)	5	4	3	0–2
Innovation and Critical Thinking (4)	4	3	2	0–1
Performance Analysis and Optimization (4)	4	3	2	0–1
Presentation and Viva Voce (4)	4	3	2	0–1
Completeness of the Report (3)	3	2	1	0

Level	Description
Excellent	Demonstrates outstanding technical knowledge, accurate implementation, effective integration, and innovative problem-solving.
Good	Demonstrates good understanding and implementation with minor errors or improvements needed.
Average	Demonstrates basic understanding and partial implementation with noticeable limitations.
Below Average	Demonstrates limited understanding, incomplete implementation, and significant technical issues.

Suggested assessment areas: ROS/ROS2, Gazebo or other robotics simulators, sensor data processing, path planning, PID control, computer vision, and robot performance evaluation, depending on the project.

Signature of the Faculty-in-charge

